Jev-Backed Function Calling: Iterative Repair with Cumulative Decision Feedback

Minhyeong Yu
Enhans
minhyeong@enhans.ai

Abstract

Function-calling models must choose a function and input values that match the user's request. Correct format alone does not ensure correct choices. We present Jev-Backed Function Calling (**JBFC**), an inference-time method that uses TypeSafe's System One (Decision) model *Jev* to check an LLM's proposed call. *Jev* checks function choice and, when a call is present, its arguments with binary questions. If a required check fails, we add the proposal and judgments to a growing history placed before the original user request. The LLM uses this feedback to revise both the function and arguments. Acceptance requires all applicable binary checks and the schema check to pass on the same proposal. The repair limit is fixed at two. Our code compares seven methods on a 3,051-example Berkeley Function Calling Leaderboard (BFCL) subset, measuring call correctness, function choice, argument correctness, time, and cost. In one run with `gpt-5-nano`, **JBFC** raises call accuracy over direct calling from 63.9% to 73.1%, mostly by avoiding unnecessary calls, at about four times the cost. The other comparisons remain to be run; they will test whether accumulated verification feedback improves calls and whether repairs justify their added cost.

1 Introduction

To carry out a user's request correctly, a function-calling system must choose the right function and provide the right argument values. A language model, called the generator, makes these choices. However, a call can have the required format and still use the wrong function or values. For example, a travel assistant may choose the wrong booking function or swap the departure and destination. A JSON schema check verifies required inputs and their types, but cannot guarantee that the call matches the user's request.

Tool-use training and evaluation have improved function-calling systems [2, 3, 1]. Even so, the generated function and arguments can still be wrong. We study whether checks from a separate decision model can guide the LLM to improve its next proposal before execution.

JBFC uses *Jev*, TypeSafe's System One model, to answer binary questions about an LLM's function choice and arguments [12]. When a check fails, we keep the proposed call and its judgments and place all earlier records before the original user request. The LLM then proposes a new call using this accumulated feedback. It can change both the function and argument values. **JBFC** accepts a proposal only when all applicable binary checks and the schema check pass on that proposal.

Earlier work already uses checks and feedback to fix calls. ToolVerifier checks tool choice and parameters [9]; ToolCritic gives feedback on tool-use errors [10]. ReAct uses reasoning, actions, and observations to guide later steps [16]. **JBFC** also carries feedback forward in the prompt, but its feedback comes from checking proposed calls before execution. Our contribution is this specific use of two *Jev* judgments and accumulated prompts, together with an evaluation of its effects and costs.

We compare **JBFC** with direct calling, schema repair, self-revision, single-check versions, and a latest-feedback version. A fixed repair limit of two controls the comparison. We measure errors fixed or introduced by repair, and keep call correctness separate from acceptance, time, and cost.

2 Related Work

Tool use and benchmarks. Gorilla studies API-call generation [2]; ToolLLM studies tool-use training and evaluation [3]. BFCL tests function calling across many settings [1]. We use its single-turn Python categories to study decisions made before a function runs. API-Bank offers dialogues with executable API calls [4]. StableToolBench uses caching and simulation to handle changing APIs [5]. These suggest broader tests beyond our BFCL subset.

Checks and feedback. Self-Refine uses self-generated feedback [6], Reflexion uses written feedback [7], and CRITIC uses feedback from tools [8]. Self-correction can turn correct answers into wrong ones [11], so we measure both fixes and new errors. ToolVerifier compares alternatives to check tool choice and parameters separately [9]. ToolCritic identifies tool-use errors and returns feedback [10]. **JBFC** checks complete proposals with a fixed external model through an API and limits repairs.

Accumulated prompts. ReAct combines reasoning with actions and observations from an environment [16]. Our method uses a similar idea of carrying earlier feedback into the next prompt. Each record contains a proposed function call and verification results, rather than an execution result. The LLM receives all records before the unchanged user request. We do not require a generated reasoning trace or run functions during repair.

***Jev* as a judge.** Li et al.'s *JEV-as-a-Judge* [15] checks finished outputs with probabilities for defined verdicts. If a verdict is uncertain, an independent, stronger judge decides.

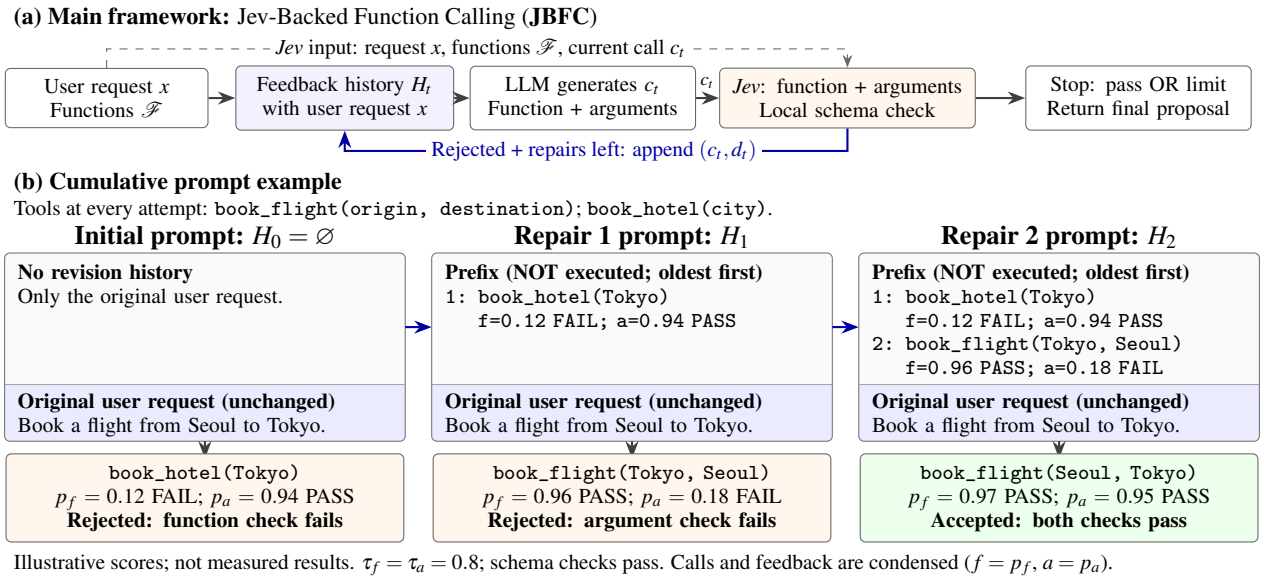


Figure 1: **JBFC** and cumulative prompt feedback. Top: the LLM reads earlier calls and judgments (H_t), then the original request (x), and generates a call (c_t) with a function and arguments. Failed checks add a record for another attempt. Bottom: growing history precedes the unchanged request; new calls appear below. Argument checks concern the proposed function. No function runs during repair.

With fixed rules for when to use the stronger judge, it lowers judging cost while keeping similar accuracy on some tasks. It has limits on difficult reasoning, misleading answer styles, and writing judged without reference information. The study does not test tool or action selection inside agents. **JBFC** checks proposed function calls and returns failed checks to the same generator for revision. We test the specific design of **JBFC**.

3 Method

3.1 Propose and Check a Call

Let x be the user request, $\mathscr{F}$ the available functions, and G the generating LLM. A proposal is $c_t = (f_t, a_t)$, with function f_t and arguments a_t, or $c_t = \varnothing$ for no call. We study single-call-or-no-call decisions.[1] Local code checks readability, function availability, required arguments, and types. Write $S(c_t) = 1$ when these checks pass.

Binary verification: JBFC. For each readable proposal, one *Jev* request checks the function choice and, when a call is present, its arguments:

$$c_t = G(H_t, x, \mathscr{F}), \tag{1}$$

$$p_f^{(t)} = J_f(x, \mathscr{F}, c_t), \qquad p_a^{(t)} = J_a(x, \mathscr{F}, c_t). \tag{2}$$

Here H_t contains earlier proposals and feedback and is initially empty. The function `Noul` question asks whether the function choice, including no call, fits the request. The argument question checks required values, their support in the request, and their parameter roles. For no call, it is skipped and $p_a^{(t)}$ is undefined. A selected function with missing arguments still receives both checks. Both must pass for a call.

[1] We treat single-turn decisions as the basic unit of function calling. Parallel calling combines multiple such decisions, while multi-turn interactions repeat them with updated context. We therefore evaluate the verification and repair loop on this basic unit and exclude parallel and multi-turn settings from this study.

3.2 Accept or Repair a Call

Acceptance requires the schema check and all required *Jev* checks to pass on the same proposal. The acceptance rule is

$$A_t = \begin{cases} S(c_t)\,\mathbf{1}[p_f^{(t)} \geq \tau_f], & c_t = \varnothing, \\ S(c_t)\,\mathbf{1}[p_f^{(t)} \geq \tau_f]\,\mathbf{1}[p_a^{(t)} \geq \tau_a], & \text{otherwise.} \end{cases} \tag{3}$$

`Noul` returns the probability of yes, with no separate confidence field [13, 14]. We start with $\tau_f = \tau_a = 0.8$, untuned defaults chosen for this experiment.

Feedback before the user request. H_t stores earlier records. The current record (c_t, d_t) pairs proposal c_t with feedback d_t (probabilities, pass/fail flags, thresholds, and local errors). No-call skips the argument check. After rejection, if repairs remain, $\|$ adds $[(c_t, d_t)]$, a list containing this one record, to the end of H_t:

$$H_{t+1} = H_t \,\|\, [(c_t, d_t)]. \tag{4}$$

The LLM reads this history before the unchanged request and can change both function and arguments. Schemas remain available; system instructions stay separate.

Algorithm 1 rechecks up to $r+1$ proposals: the first plus repairs (Python default $r = 3$; $r = 2$ in experiments). Its `Prefix` function places JSON records, oldest first, before

Algorithm 1 JBFC: verification and prompt repair

Require: Request x, functions $\mathscr{F}$, generator G, *Jev* J, repair limit r, thresholds τ_f, τ_a

$H \leftarrow [\,]$
for $t = 0, \ldots, r$ **do**
  $c \leftarrow G(\text{Prefix}(H, x), \mathscr{F})$
  $(s, e, \text{readable}) \leftarrow \text{SchemaCheck}(c, \mathscr{F})$
  $p_f, p_a \leftarrow \text{null}, \text{null}$
  if readable **then**
    $Q \leftarrow \{\text{function}\}$
    if $c \neq \varnothing$ **then** $Q \leftarrow Q \cup \{\text{arguments}\}$
    $(p_f, p_a) \leftarrow J(x, \mathscr{F}, c, Q)$
  end if
  $A \leftarrow \text{Accept}(s, c, p_f, p_a, \tau_f, \tau_a)$ ▷ Eq. 3
  $d \leftarrow \text{Feedback}(e, p_f, p_a, \tau_f, \tau_a, c)$
  if $A = 1$ **or** $t = r$ **then break**
  $H \leftarrow H \,\|\, [(c, d)]$
end for
return c

unchanged original messages. Empty history adds no prefix. Records include round numbers and mark proposals as not executed. Here $s = S(c)$ and e lists local errors. Readable proposals receive *Jev* checks even if their schema fails; unreadable ones receive local feedback. *Jev* never receives history. Unasked p_a stays null. Null required probabilities cannot pass the gate; invalid API responses stop the run. Reference answers stay out of model requests. API errors stop the run after transport retries, separate from repairs.

3.3 Final Proposal

When a proposal passes all applicable checks, we return it. If the repair limit is reached first, we return the last proposal even if it fails a check (`execute_last`). We score the final proposal against the benchmark answers without running the function.

4 Experiments

4.1 Data and Evaluation

We prepare 3,051 BFCL examples from seven categories at repository revision `6ea57973c7a6`[2]. Table 1 shows the example counts and results. We use all examples and tune nothing on them: the repair limit is fixed at $r = 2$ and the *Jev* thresholds at 0.8.

For categories with reference calls, the original Python abstract syntax tree (AST) checker tests function names and arguments against allowed answers. Irrelevance categories require no call. Live relevance checks only call presence, so we exclude it from main accuracy and Brier scores. The main score is a micro-average over 3,035 examples; category scores show what the larger live categories may hide.

BFCL's "multiple" categories offer several functions but no parallel calls; we exclude parallel, multi-turn, web-search, memory, and execution categories. AST checks compare call structure with annotated alternatives, not execution success.

[2]Full revision: `6ea57973c7a6097fd7c5915698c54c17c5b1b6c8`. The saved setup file lists source URLs, file hashes, and format changes; each run's manifest lists the example IDs.

Category	Examples	Direct	**JBFC**
Simple Python	400	91.2	91.2
Multiple	200	91.0	91.0
Irrelevance	240	25.4	**72.9**
Live simple	258	73.6	73.6
Live multiple	1053	68.3	**70.8**
Live irrelevance	884	47.6	**63.5**
Live relevance[†]	16	87.5	87.5
Total	3051	63.9	**73.1**

Table 1: BFCL example counts and accuracy (%) on all examples, from one run with repair limit $r = 2$. Generator: `gpt-5-nano`. Bold marks the higher accuracy when the two differ. [†]Live relevance checks only whether a call is present; total accuracy excludes these 16 cases.

Method	Call	Func.	Args.	Sec.	USD/1k
Direct	–	–	–	–	–
Schema repair	–	–	–	–	–
Self-refine	–	–	–	–	–
JBFC-F	–	–	–	–	–
JBFC-A	–	–	–	–	–
JBFC-L	–	–	–	–	–
JBFC	–	–	–	–	–

Table 2: Methods on all BFCL examples, from one run with repair limit $r = 2$. Call and function accuracy (%) exclude live relevance. Argument accuracy (%) uses calls with the correct function and argument labels. – means unavailable or unrun.

4.2 Models and Methods

The generator is the Azure OpenAI deployment `gpt-5-nano` (`gpt-5-nano-2025-08-07`), with reasoning effort `none` and at most 4,096 output tokens; *Jev* is pinned to `jev-1.13.0`. We use Chat Completions and System One through an APIM gateway and convert BFCL schemas to JSON Schema with API-safe function names. Each method runs once per example with fixed prompts. Generations cut off at the output limit or blocked by the provider's filter are unreadable proposals and count as wrong.

We compare seven methods, sharing one initial proposal per example. *Direct* uses the initial proposal; *Schema-only* repairs schema errors. *Self-refine* uses generic self-review in all r repairs, not the full Self-Refine method [6]. **JBFC-F** and **JBFC-A** obtain applicable `Noul` judgments but expose and require only the function or argument verdict, plus schema checks. **JBFC-A** accepts no call on schema checks alone. **JBFC** requires all applicable checks. **JBFC-L** shares its settings and initial proposal but shows only the latest rejected proposal and feedback. Other repair methods show all earlier records before the request. Costs can differ. With $r = 2$, repair methods make at most three proposals, and the **JBFC** and **JBFC-L** prompts differ only in the second repair.

Repair limits. Table 3 varies r for **JBFC**. Neither model sees r and nothing runs before the end, so a shorter trajectory is the start of a longer one. We continue the $r = 2$ trajectories

r	Call	Func.	Repairs	Exhaust.	Sec.	USD/1k
1	–	–	–	–	–	–
2	–	–	–	–	–	–
3	–	–	–	–	–	–
5	–	–	–	–	–	–
7	–	–	–	–	–	–

Table 3: **JBFC** on all BFCL examples by repair limit r, cut from the run of Table 2 extended to $r = 7$, so $r = 2$ matches it. Accuracy (%) excludes live relevance; Exhaust.: examples reaching the limit without passing (%). – means unrun.

of Table 2 that reach the limit up to $r = 7$ and cut them at each limit.

4.3 Measures, Cost, and Uncertainty

We report final call accuracy, function accuracy, and argument accuracy given the correct function, each with its denominator; comparing initial and final candidates shows fixes and new errors. Acceptance is distinct from correctness: we also record coverage, accepted precision, false acceptance and rejection, repairs, API errors, and Brier scores where labels exist.

Costs count all generation and verification work, including the growing history, with the shared initial call charged to each method; times are mean API seconds without retry waits. Missing usage or prices stay unavailable, not zero. Prices per million tokens are \$0.05/\$0.005/\$0.40 for uncached/cached/output `gpt-5-nano` tokens and \$0.042 for *Jev* input (output free), without gateway fees.

We compare repairs with Direct and **JBFC** with **JBFC-L**, matching examples and initial proposals, and bootstrap example IDs 2,000 times with seed 1729 for 95% intervals. Tables 2 and 3 remain unrun; partial or synthetic runs cannot fill the tables.

4.4 Results

Table 1 compares Direct with **JBFC**. **JBFC** raises overall accuracy from 63.9% to 73.1% (paired difference 9.2 points, 95% interval 8.1–10.4). It turns 322 wrong initial proposals into correct ones and 42 correct ones into wrong ones. It uses 0.76 repairs per example on average, and 32.7% of examples reach the limit without passing. Mean API time rises from 1.37 to 3.03 seconds, and cost from \$0.057 to \$0.224 per 1,000 requests.

Why no-call cases gain most. Every Irrelevance example offers one unrelated function, and Direct calls it in 179 of 240 cases; a second Direct run gave a similar 27.1% accuracy. *Jev*'s function question separates these calls from correct no-calls: its median probability is 0.10 for the calls and 0.95 for the 61 initial no-calls. **JBFC** rejects 165 of the 179 calls, and after the feedback the generator withholds the call in 114 of them. When *Jev* still rejects such a final no-call, `execute_last` returns it anyway. No initial no-call is lost; the other 65 errors are 14 accepted calls and 51 cases where the generator kept calling. Live irrelevance offers 2.3 functions on average and real user requests. There, *Jev* separates less sharply (median 0.53 for unnecessary calls and 0.89 for correct no-calls), the generator drops only 164 of 439 rejected calls, and 24 correct no-calls become calls after a false rejection. In categories with reference calls, **JBFC** rejects 338 of 455 wrong initial calls but fixes only 44 and breaks 18. A binary verdict says that a call is wrong, not which call is right; for an unnecessary call, withholding it is the obvious repair.

4.5 Study Limits

The verifier can accept a wrong call or share the generator's mistake, and repair can introduce errors. Accumulated feedback can reinforce a mistake or distract the LLM, and it grows with each repair. Binary verdicts do not explain how to fix an error, and higher acceptance alone does not show improvement.

The study uses one model pair, mostly English requests, and single-turn calls. Public examples may overlap with training data. We keep ten documented annotation inconsistencies in the main score. In six, reference values contradict declared types, so schema-checked methods cannot accept a correct call. We fix $r = 2$, the thresholds, and reasoning effort `none` without tuning; other settings may change the results, and one common limit may suit some methods better than others. A single run does not show run-to-run variation. Further studies should tune each method separately and test more models and domains.

5 Conclusion

JBFC uses *Jev*'s function and argument judgments as accumulated prompt feedback. After a failed check, earlier proposals and judgments appear before the original user request, allowing the LLM to revise the complete call. A fixed repair limit of two bounds this process. The code separates correctness, acceptance, and cost. In one run with `gpt-5-nano`, **JBFC** raised accuracy from 63.9% to 73.1% at about four times the cost, mostly by withholding unnecessary calls. The comparison with the other methods remains to be run; it will test whether accumulated feedback, beyond verification alone, explains this gain and whether it justifies the added work.

Acknowledgements

This work was created as part of the No-More-Slops group activities at Pseudo Lab.

References

[1] Shishir G. Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The Berkeley Function Calling Leaderboard (BFCL): From Tool Use to Agentic Evaluation of Large Language Models. In *International Conference on Machine Learning*, PMLR 267, pp. 48371–48392, 2025. https://proceedings.mlr.press/v267/patil25a.html.

[2] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large Language Model Connected with Massive APIs. In *Advances in Neural Information Processing Systems*, 37:126544–126565, 2024. https://arxiv.org/abs/2305.15334.

[3] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2307.16789.

[4] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Empirical Methods in Natural Language Processing*, pp. 3102–3116, 2023. https://aclanthology.org/2023.emnlp-main.187/.

[5] Zhicheng Guo, Sijie Cheng, Hao Wang, Shihao Liang, Yujia Qin, Peng Li, Zhiyuan Liu, Maosong Sun, and Yang Liu. StableToolBench: Towards Stable Large-Scale Benchmarking on Tool Learning of Large Language Models. In *Findings of ACL*, pp. 11143–11156, 2024. https://aclanthology.org/2024.findings-acl.664/.

[6] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems*, 36:46534–46594, 2023. https://arxiv.org/abs/2303.17651.

[7] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Advances in Neural Information Processing Systems*, 36:8634–8652, 2023. https://arxiv.org/abs/2303.11366.

[8] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Nan Duan, and Weizhu Chen. CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2305.11738.

[9] Dheeraj Mekala, Jason Weston, Jack Lanchantin, Roberta Raileanu, Maria Lomeli, Jingbo Shang, and Jane Dwivedi-Yu. ToolVerifier: Generalization to New Tools via Self-Verification. *arXiv:2402.14158*, 2024. https://arxiv.org/abs/2402.14158.

[10] Hassan Hamad, Yingru Xu, Liang Zhao, Wenbo Yan, and Narendra Gyanchandani. ToolCritic: Detecting and Correcting Tool-Use Errors in Dialogue Systems. *arXiv:2510.17052*, 2025. https://arxiv.org/abs/2510.17052.

[11] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. Large Language Models Cannot Self-Correct Reasoning Yet. In *International Conference on Learning Representations*, 2024. https://arxiv.org/abs/2310.01798.

[12] TypeSafe. Models: *Jev* and the System One API. Technical documentation, 2026. Accessed October 15, 2026. https://docs.typesafe.ai/models.

[13] TypeSafe. Noul. Technical documentation, 2026. Accessed October 17, 2026. https://docs.typesafe.ai/primitives/noul.

[14] TypeSafe. Confidence. Technical documentation, 2026. Accessed October 17, 2026. https://docs.typesafe.ai/confidence.

[15] Yubo Li, Yidi Miao, Ramayya Krishnan, and Rema Padman. JEV-as-a-Judge: Accept When Confident, Escalate When Unsure. *arXiv:2609.26550*, version 5, October 9, 2026. https://arxiv.org/abs/2609.26550v5.

[16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations*, 2023. https://arxiv.org/abs/2210.03629.